



UNIVERSIDAD TÉCNICA PARTICULAR DE LOJA
La Universidad Católica de Loja

CARRERA DE COMPUTACIÓN

Intregantes: Arturo Angamarca, Mateo Peralta, Juan Diego Guerrero.

PROGRAMACIÓN ORIENTADA A OBJETOS

Bimestre: Segundo Bimestre

Profesor: Ing. Ramiro Ramírez

Fecha de entrega: 25 de Junio del 2026

1. Análisis del Problema

1. ¿Qué problemas puede ocasionar un sistema altamente acoplado?

Un sistema altamente acoplado (donde las clases dependen excesivamente y de forma directa unas de otras) genera lo que en ingeniería de software se conoce como un diseño rígido o "código espagueti". Los principales problemas que ocasiona son:

- **Efecto dominó (Fragilidad):** Al modificar una línea de código o una clase, es muy probable que se rompan otras partes del sistema en cascada porque dependen fuertemente de esa implementación.
- **Dificultad de mantenimiento:** Entender cómo funciona un solo módulo requiere leer y entender todo el sistema, lo que hace que buscar y arreglar errores (debugging) sea lento, frustrante y costoso.
- **Imposibilidad de reutilización:** Como las clases están amarradas a herramientas específicas (ej. un Jugador amarrado a una BaseDatosMySQL), no puedes extraer esa clase para usarla en otro proyecto sin llevarte todo el código sobrante con ella.
- **Difícil de testear:** Es casi imposible realizar pruebas unitarias (Unit Testing) porque no se puede aislar una clase de sus dependencias para probarla de forma individual.

2. ¿Qué ventajas ofrece SOLID? Aplicar los principios SOLID transforma un sistema rígido en una arquitectura flexible, escalable y profesional. Sus principales ventajas son:

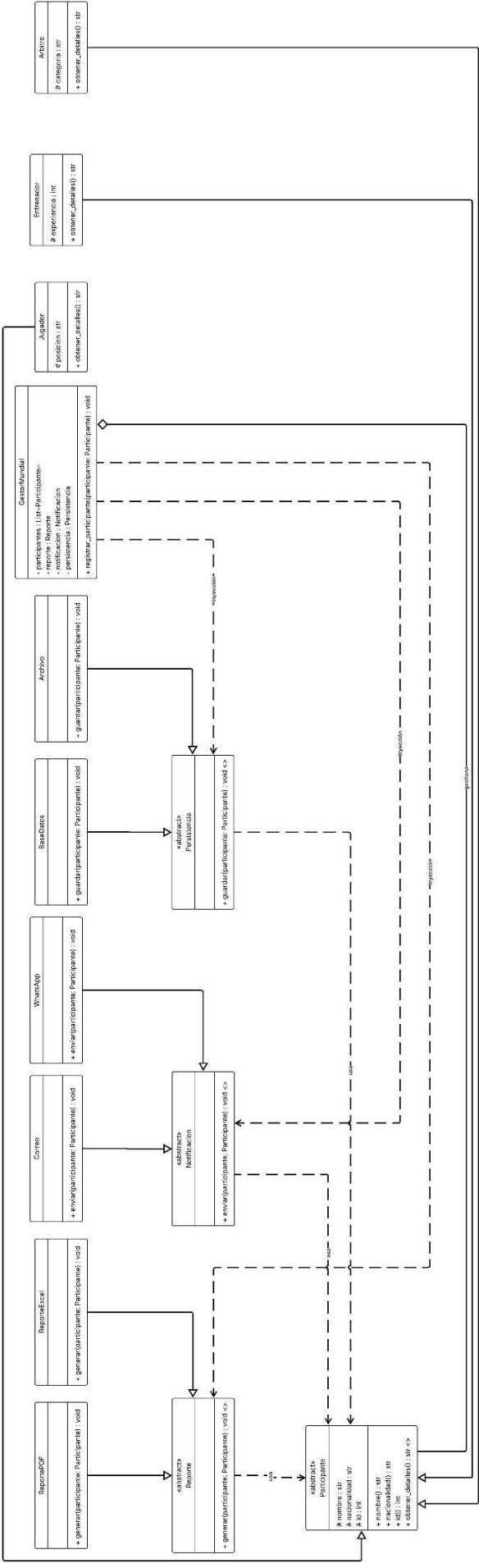
- **Escalabilidad ágil (Extensibilidad):** Permite agregar nuevas funcionalidades (como los nuevos integrantes Médico o Comentarista) simplemente creando nuevas clases, sin necesidad de modificar o romper el código que ya está en producción.
- **Alta mantenibilidad:** El código resulta limpio, modular y predecible. Cada clase tiene un propósito claro, lo que facilita que nuevos programadores entiendan el proyecto rápidamente.
- **Bajo acoplamiento y Alta cohesión:** Los módulos se vuelven independientes al comunicarse a través de interfaces (abstracciones) en lugar de depender de clases concretas.
- **Facilidad para pruebas:** Al tener responsabilidades únicas y dependencias invertidas, es muy sencillo crear "mocks" (objetos simulados) para probar cada pieza del software de forma automatizada.

3. ¿Qué principio consideran más importante y por qué?

Consideramos que el principio más importante es el Principio de Responsabilidad Única (SRP - Single Responsibility Principle), ya que actúa como el cimiento fundamental de todos los demás principios.

¿Por qué? Si una clase hace demasiadas cosas y tiene múltiples responsabilidades (no cumple SRP), será matemáticamente imposible mantenerla cerrada a modificaciones (violando el principio OCP), probablemente forzará la creación de interfaces gigantes y genéricas (violando el principio ISP) y acumulará múltiples dependencias que dificultarán la abstracción (violando el principio DIP). Al aplicar correctamente el SRP, el código se divide naturalmente en piezas pequeñas, enfocadas y cohesivas, lo que hace que implementar el resto de los principios SOLID sea un proceso casi automático.

2. Diagrama UML



3. Código en Python

```
from abc import ABC, abstractmethod

class Participante(ABC):
    def __init__(self, nombre: str, nacionalidad: str, id_participante: int):
        self._nombre = nombre
        self._nacionalidad = nacionalidad
        self._id = id_participante

    @property
    def nombre(self) -> str:
        return self._nombre

    @property
    def nacionalidad(self) -> str:
        return self._nacionalidad

    @property
    def id(self) -> int:
        return self._id

    @abstractmethod
    def obtener_detalle(self) -> str:
        pass

class Jugador(Participante):
    def __init__(self, nombre: str, nacionalidad: str, id_participante: int, posicion: str):
        super().__init__(nombre, nacionalidad, id_participante)
        self._posicion = posicion

    def obtener_detalle(self) -> str:
        return f"Jugador: {self.nombre} ({self.nacionalidad}) - Posición: {self._posicion}"

class Entrenador(Participante):
    def __init__(self, nombre: str, nacionalidad: str, id_participante: int, experiencia: int):
        super().__init__(nombre, nacionalidad, id_participante)
        self._experiencia = experiencia

    def obtener_detalle(self) -> str:
        return f"Entrenador: {self.nombre} - Experiencia: {self._experiencia} años"

class Arbitro(Participante):
    def __init__(self, nombre: str, nacionalidad: str, id_participante: int, categoria: str):
        super().__init__(nombre, nacionalidad, id_participante)
        self._categoria = categoria

    def obtener_detalle(self) -> str:
        return f"Árbitro: {self.nombre} - Categoría: {self._categoria}"

class Reporte(ABC):
    @abstractmethod
    def generar(self, participante: Participante):
        pass

class Notificacion(ABC):
    @abstractmethod
    def enviar(self, participante: Participante):
```

```

        pass

class Persistencia(ABC):
    @abstractmethod
    def guardar(self, participante: Participante):
        pass

class ReportePDF(Reporte):
    def generar(self, participante: Participante):
        print(f"[PDF] Generando documento PDF para: {participante.nombre}")

class ReporteExcel(Reporte):
    def generar(self, participante: Participante):
        print(f"[EXCEL] Generando hoja de cálculo para: {participante.nombre}")

class Correo(Notificacion):
    def enviar(self, participante: Participante):
        print(f"[EMAIL] Enviando correo a {participante.nombre}")

class WhatsApp(Notificacion):
    def enviar(self, participante: Participante):
        print(f"[WHATSAPP] Enviando mensaje a {participante.nombre}")

class BaseDatos(Persistencia):
    def guardar(self, participante: Participante):
        print(f"[BD] Ejecutando INSERT en base de datos para ID: {participante.id}")

class Archivo(Persistencia):
    def guardar(self, participante: Participante):
        print(f"[ARCHIVO] Guardando en TXT local los datos de ID: {participante.id}")

class GestorMundial:
    def __init__(self, reporte: Reporte, notificacion: Notificacion, persistencia:
Persistencia):
        self.participantes = []
        self.reporte = reporte
        self.notificacion = notificacion
        self.persistencia = persistencia

    def registrar_participante(self, participante: Participante):
        print("\n--- Procesando Registro ---")
        print(participante.obtener_detalle())
        self.participantes.append(participante)

        self.persistencia.guardar(participante)
        self.reporte.generar(participante)
        self.notificacion.enviar(participante)
        print("-----\n")

if __name__ == "__main__":
    gestor = GestorMundial(ReportePDF(), WhatsApp(), BaseDatos())

    while True:
        print("¿Qué desea hacer?")
        print("1. Ingresar Jugador")
        print("2. Ingresar Entrenador")
        print("3. Ingresar Árbitro")
        print("4. Salir")
        opcion = input("Seleccione una opción: ")

```

```

if opcion == "4":
    break

if opcion in ["1", "2", "3"]:
    nombre = input("Nombre: ")
    nacionalidad = input("Nacionalidad: ")
    id_participante = int(input("ID: "))

    nuevo_participante = None

    if opcion == "1":
        posicion = input("Posición: ")
        nuevo_participante = Jugador(nombre, nacionalidad, id_participante, posicion)
    elif opcion == "2":
        exp = int(input("Años de experiencia: "))
        nuevo_participante = Entrenador(nombre, nacionalidad, id_participante, exp)
    elif opcion == "3":
        cat = input("Categoría: ")
        nuevo_participante = Arbitro(nombre, nacionalidad, id_participante, cat)

    if nuevo_participante:
        gestor.registrar_participante(nuevo_participante)
    else:
        print("Opción inválida.")

```

4. Capturas del código en ejecución

```

¿Qué desea hacer?
1. Ingresar Jugador
2. Ingresar Entrenador
3. Ingresar Árbitro
4. Salir
Seleccione una opción: 1
Nombre: Gonzalo PLata
Nacionalidad: Ecuatoriana
ID: 1231
Posición: Extremo

--- Procesando Registro ---
Jugador: Gonzalo PLata (Ecuatoriana) - Posición: Extremo
[BD] Ejecutando INSERT en base de datos para ID: 1231
[PDF] Generando documento PDF para: Gonzalo PLata
[WHATSAPP] Enviando mensaje a Gonzalo PLata
-----

```

```

¿Qué desea hacer?
1. Ingresar Jugador
2. Ingresar Entrenador
3. Ingresar Árbitro
4. Salir
Seleccione una opción: 2
Nombre: Zinedin Zidane
Nacionalidad: Francesa
ID: 3213
Años de experiencia: 10

--- Procesando Registro ---
Entrenador: Zinedin Zidane - Experiencia: 10 años
[BD] Ejecutando INSERT en base de datos para ID: 3213
[PDF] Generando documento PDF para: Zinedin Zidane
[WHATSAPP] Enviando mensaje a Zinedin Zidane
-----

```

```

¿Qué desea hacer?
1. Ingresar Jugador
2. Ingresar Entrenador
3. Ingresar Árbitro
4. Salir
Seleccione una opción: 3
Nombre: Pierluigi Collina
Nacionalidad: Italiana
ID: 2342
Categoría: Elite-Fifa

--- Procesando Registro ---
Árbitro: Pierluigi Collina - Categoría: Elite-Fifa
[BD] Ejecutando INSERT en base de datos para ID: 2342
[PDF] Generando documento PDF para: Pierluigi Collina
[WHATSAPP] Enviando mensaje a Pierluigi Collina
-----

```